
GLOBAL COMMODITY INSIGHTS LTD

Commodity Market Intelligence Platform

Individual Reflective Journal

Internship	London Success Academy, Applied Data Engineering Internship
Project	Commodity Market Intelligence Platform
Role	Data Engineering Team, Cloud Platform and Analytics
Platform	Snowflake Cloud Analytics (COMMODITY_WH)
Period	June 2026, Weeks 1 to 3

1. Introduction

This reflective journal documents my personal experience working on the Commodity Market Intelligence Platform as part of the London Success Academy Applied Data Engineering Internship. Reflection matters in this field. Completing a technical task is only half the job; understanding why each decision was made, and what should change next time, is the other half.

This was my first attempt at building a full data pipeline, from raw CSV files through to executive-ready analytics on a cloud platform, and I learned more in three weeks than I expected to.

2. Project Overview and My Role

Our team was tasked with designing and implementing the first version of a cloud-based analytics platform for Global Commodity Insights Ltd. The company needed to move away from manual spreadsheet-based reporting and towards a trusted, centralised data infrastructure.

My primary responsibilities within the team covered:

- Designing the Snowflake environment architecture (warehouses, schemas, file formats, stages)
- Writing the data loading scripts (COPY INTO) and staging transformation logic
- Building the data augmentation layer with statistical features (moving averages, volatility, lag, drawdown)
- Implementing Snowflake-specific advanced features: Stream, Task, Time Travel, Stored Procedure, Zero-copy Clone
- Producing the business insight queries and executive views

The work involved juggling several concerns at once: getting the data right (quality checks), writing SQL that does not break on bad input (defensive casting, OHLC validation), keeping the architecture clear (layered schemas), and making sure the output answers real business questions.

3. What Went Well

3.1 Architectural Design

Choosing a four-layer architecture (RAW, STAGING, ANALYTICS, AUDIT) turned out to be one of our best calls. It meant that:

1. The original source data was always preserved intact in the RAW layer, giving us a reliable rollback point
2. Business logic transformations were isolated in the STAGING layer, making them easy to modify without touching raw data

-
3. The ANALYTICS layer contained only clean, enriched, business-ready data that analysts could query with confidence
 4. Quality events were captured in the AUDIT layer with full historical context

I had read about this separation of concerns before, but watching it stop errors from cascading downstream gave me a much better appreciation of why teams use it.

3.2 Defensive SQL and Type Safety

We used TRY_TO_DATE(), TRY_TO_DOUBLE() and TRY_TO_NUMBER() throughout staging on purpose. Instead of letting one malformed value kill the pipeline, these functions return NULL for anything unparseable, and we filter those rows out explicitly. The result was a pipeline that shrugged off edge cases in the source files.

The OHLC logic check was a small thing I was pleased with. It validates four conditions at once (LOW <= OPEN, LOW <= CLOSE, CLOSE <= HIGH, OPEN <= HIGH), which is standard in financial data work but took some care to get right in SQL.

3.3 Snowflake-Native Differentiators

Choosing Snowflake over the recommended BigQuery let us use features that most teams would not have access to:

- Streams and Tasks: TASK_REFRESH_GOLD_STAGING watches the RAW stream and refreshes staging on a schedule without anyone touching it. That is an actual pipeline rather than a one-time script.
- Time Travel: querying a table as it was minutes earlier (AT OFFSET => -300) gave us a built-in audit capability at no extra cost.
- Zero-copy Clone: STG_GOLD_MARKET_DEV is an instant copy of staging, so the team can experiment without putting production data at risk.
- Stored Procedures: wrapping the quality checks in SP_RUN_DQ_CHECKS() means they can be re-run on demand, or hooked into the task schedule, without running scripts one by one. We later chained these into a full Task DAG so that a single new row in the raw table flows automatically through staging, the fact rebuild, the quality checks and finally the dashboard, with no manual step. Watching that run end to end was the moment the project stopped being a set of scripts and became an actual platform.

3.4 Business Insight Depth

We delivered ten insights against the five required, but the part I found most rewarding came later, when a reviewer pushed us to move from description to hypotheses. Instead of just reporting that oil fell in 2015, we asked sharper questions: is oil structurally more volatile than gold, does a strong dollar really push gold down, does holding both commodities actually reduce risk? Testing each one in SQL produced answers that were more useful than any single price statistic, and two of them disproved assumptions we had started with. That shift, from stating facts to testing claims, was the moment the project started to feel like real analysis rather than reporting.

The thing I would single out: the persistent AUDIT.DQ_LOG table. Instead of one-off quality checks, we built a queryable audit trail that supports governance, SLA monitoring and quality trends over time.

4. What Did Not Go Well

4.1 DIM_DATE Dependency Discovered Late

The biggest technical mistake in our first build was forgetting to create DIM_DATE before running DATA_AUGMENTATION.sql. The fact tables join to that dimension for every date-derived column (YEAR, QUARTER, MONTH_NAME, IS_WEEKDAY and so on), so without it every dimension column in both fact tables would have come out NULL.

We caught it in a validation review rather than during development, which means it could easily have reached a demo environment. The lesson: map script dependencies before running anything, and write the execution order down somewhere everyone can see it.

Next time I would write an execution manifest on day one: a numbered list of scripts and their prerequisites, so nothing runs before the things it depends on.

4.2 Oil CSV Volume Format Not Anticipated

The oil_price.csv file stores volume as abbreviated strings such as "275.86K" rather than plain integers. In our first load this left TRADE_VOLUME NULL for every oil record until we added the REPLACE() and TRY_TO_NUMBER() handling in staging. We found it in the quality review, but it really belonged to the data understanding phase.

The root cause was simple: we started writing loading scripts before properly inspecting the source files. Looking at sample values in every column up front would have surfaced this immediately and saved the rework.

4.3 The Gold Dataset's 81 Columns Were Underutilised

The gold dataset carries 81 columns spanning several metals, ETFs, currencies and equity indices, yet our analysis stayed narrow. We used only 11 of the 81 columns in staging, mostly gold OHLCV and a few market indices.

Looking back, columns like SF_Price (silver), PLT_Price (platinum) and EU_Price (the euro) could have powered a multi-commodity correlation story that would have strengthened the executive insights considerably. In a second sprint this is where I would start.

4.4 Missing a Formal Data Dictionary

Several gold column names need decoding (EG_ is Euro Gold, OF_ is Oil Futures, PLD_ is Palladium). Without a data dictionary at the start, team members occasionally misread what a column meant.

A simple reference table mapping each prefix to its full name and type would have removed the ambiguity and helped anyone joining the project later.

5. Improvements I Would Make Next Time

5.1 Technical Process

Area	Specific Improvement
Execution Order	Write a numbered script manifest with dependencies before any SQL is written
Data Understanding	Profile every column (sample values, formats, ranges) in a dedicated session before writing loading scripts
Data Dictionary	Build a reference table for all column name prefixes on day one of the project
Pipeline Automation	Chain the four stages (load, quality, staging, analytics) into one Task pipeline from the start
Volume Data	Always check for string-encoded numeric columns in source files before designing COPY INTO statements
Column Utilisation	Map all 81 gold columns to business questions in the data understanding phase, not the insight phase

5.2 Team Collaboration

If I were to lead this project in a future engagement, I would introduce three structural changes to the team process:

- Daily stand-ups with explicit blockers. We would have caught the DIM_DATE dependency earlier if everyone shared in-progress work each morning. Fifteen minutes covering what was finished, what is in flight and what is stuck would have surfaced it before it became an error.
- Code ownership per file. One named owner per SQL file would cut merge conflicts and make it obvious who reviews each change.
- Peer review for every script. No SQL file should count as done until a second person has checked it for correctness, readability and schema alignment. Professional teams treat this as routine.

6. Personal Learning and Growth

6.1 Technical Skills Acquired

Prior to this internship, my SQL experience was limited to basic SELECT, JOIN, and GROUP BY queries. Over the course of this project I gained hands-on experience with:

- Window functions (LAG, LEAD, FIRST_VALUE, LAST_VALUE, AVG OVER ROWS BETWEEN), used heavily in the augmentation layer for moving averages and volatility
- Defensive casting with TRY_TO_DATE, TRY_TO_DOUBLE and TRY_TO_NUMBER, paired with explicit NULL filtering
- Deduplication with ROW_NUMBER() OVER PARTITION BY, using QUALIFY or WHERE RN = 1
- MERGE statements for upserts in the Stream-driven Task pipeline
- Snowflake-specific tooling: GENERATOR, SEQ4, QUALIFY, VALIDATE, Time Travel, Zero-copy Clone, Streams and Tasks

6.2 Business and Professional Skills

Equally important were the non-technical lessons:

- Asking "what business question am I answering?" before writing SQL, a habit the internship brief pushed and one that has stuck
- Data engineering runs on trust as much as correctness; a stakeholder who doubts the data will not use the platform, however clever the architecture
- Documentation is part of the deliverable; unlabelled code and silent assumptions become someone else's debt
- Presenting findings in business terms (trends, risks, opportunities) rather than technical ones takes real practice

6.3 Mindset Shifts

The biggest shift for me was moving from "make it work" to "make it trustworthy". In earlier coursework, success meant the query ran. Here, success meant a stakeholder could lean on the answer.

That showed up in small decisions: logging every quality result instead of discarding it, keeping raw data instead of overwriting it, and accepting slower defensive typing in staging. Those habits are what separate professional work from coursework.

What I am taking away is not any single function or feature. It is the realisation that data engineering is judged by the decisions it makes possible, not by how clever the code looks.

7. Conclusion

The project delivered everything it set out to: a Snowflake environment with a four-layer architecture, both datasets loaded and profiled, twelve passing quality checks backed by a persistent audit log, and a set of executive views. Most importantly, it moved beyond description into eight tested hypotheses on volatility, correlation and risk. The platform works, the data can be trusted, and it now answers real business questions rather than just storing numbers.

The improvement areas I have named (earlier dependency mapping, deeper upfront profiling, a data dictionary, structured peer review) are not failures. They are what a first real engineering project is supposed to teach, and writing them down honestly is part of the job.

I am pleased with what the team built, and even more with the fact that I can now explain why each piece was built the way it was, and what I would change next time.

Submitted by: Ajay Khadke (Data Engineering Team)